

GRA: Графовая архитектура рассуждений для роевых конструкций языковых моделей

bitsoev oleg

20 июня 2026 г.

Аннотация

Большие языковые модели (LLM) демонстрируют впечатляющие индивидуальные возможности, однако организация их сотрудничества в виде роёв остаётся открытой задачей. Мы представляем **GRA** (Graph-based Reasoning Architecture) — фреймворк для построения и управления роями LLM с помощью структурированных динамических топологий. В GRA каждый агент-модель является узлом направленного ациклического графа; рёбра задают протоколы обмена, зависимости задач и пути рассуждений. Роевая конструкция описывается декларативно на предметно-ориентированном языке, что позволяет автоматически генерировать планы выполнения, обеспечивать отказоустойчивость и переконфигурировать систему в реальном времени. Мы оценили GRA на задачах многозвенного вопросно-ответного поиска, синтеза кода и дебатов. Результаты показывают, что графовые рои превосходят статические ансамбли агентов по точности (+12%), снижают задержку (-18%) и демонстрируют большую устойчивость. Код и эталонные тесты доступны по адресу <https://github.com/qqewq/GRA-LLM-Swarm-Constructs>.

1 Введение

Стремительное развитие LLM позволило создать одномодельные системы, способные отвечать на вопросы, генерировать код и вести диалог. Однако сложные задачи часто требуют разделения труда, когда специализированные агенты сотрудничают, спорят или голосуют для достижения консенсуса [1, 2]. Существующие мультиагентные фреймворки (AutoGen, CrewAI, MetaGPT) обычно опираются на линейные цепочки или жёсткие иерархические структуры, что ограничивает их адаптивность и масштабируемость.

Мы предлагаем **GRA** — графовую архитектуру рассуждений, в которой LLM-агенты представлены вершинами направленного ациклического графа (DAG). Топология графа — это не фиксированный конвейер, а *роевая конструкция* — параметризуемый шаблон, способный эволюционировать во время выполнения. Ключевые инновации:

- **Декларативный графовый язык** для описания ролей агентов, промптов и зависимостей.
- **Динамический планировщик**, распараллеливающий независимые ветви и изменяющий граф на лету (например, порождающий суб-рои).
- **Механизмы отказоустойчивости**: возврат к предыдущему состоянию, альтернативные пути.

Далее статья построена так: раздел 2 — обзор литературы; раздел 3 — детали архитектуры GRA; раздел 4 — язык роевых конструкций; раздел 5 — экспериментальные результаты; раздел 6 — заключение и будущие направления.

2 Обзор литературы

LLM-агенты. Инструменты вроде LangChain [3], AutoGPT и BabyAGI популяризовали идею автономных LLM-агентов, которые планируют и выполняют действия. Обычно они реализуют цикл одного агента с доступом к инструментам. Наша работа распространяет этот подход на мультиагентные рои.

Совместная работа нескольких агентов. Недавние системы, такие как AutoGen [4], CrewAI и MetaGPT, организуют агентов в заранее заданные сценарии общения. Координация в них, как правило, последовательная или основана на централизованной шине сообщений. GRA отличается явным использованием графовой структуры, которая выражает произвольный частичный порядок и оптимизируется для параллелизма и восстановления после сбоев.

Графовые рассуждения. Графовые нейронные сети и графы знаний применялись для улучшения рассуждений LLM [5]. Мы фокусируемся на *архитектурном* использовании графов для координации вызовов LLM, а не на рассуждениях на уровне эмбедингов.

3 Архитектура GRA

3.1 Основные абстракции

Фреймворк GRA состоит из трёх слоёв:

1. **Слой агентов:** экземпляры LLM, каждый со своим системным промптом, параметрами модели и опциональными инструментами.
2. **Графовый слой:** направленный ациклический граф $G = (V, E)$, где $v \in V$ — узел, представляющий вызов агента, а ребро $e = (u, v)$ означает, что выход u подаётся на вход v .
3. **Оркестратор:** отвечает за разбор роевой конструкции, построение начального графа, планирование узлов и обработку событий времени выполнения (ошибки, таймауты, завершения задач).

3.2 Динамическое планирование

Узлы выполняются, как только удовлетворены их зависимости. Поскольку несколько ветвей могут выполняться одновременно, оркестратор использует пулы потоков или асинхронный ввод-вывод для максимальной пропускной способности. При сбое узла (например, некорректный JSON от LLM) оркестратор может:

- Повторить попытку с уточнённым промптом.
- Активировать резервный узел, соединённый дополнительным ребром.
- Вернуться назад и перепланировать подграф с помощью облегчённого LLM-планировщика.

3.3 Управление памятью и контекстом

Каждый агентский узел поддерживает локальное окно контекста. Чтобы избежать переполнения, GRA использует шлюзы суммаризации на длинных цепочках зависимостей, управляемые свойствами рёбер (например, `max_token_budget`).

4 Язык роевых конструкций

Мы разработали декларативный язык на основе YAML для описания роевой конструкции без написания кода на Python. Пример конструкции для дебатов трёх агентов:

```
name: DebateSwarm
entry: proponent
nodes:
  - id: proponent
    agent: gpt-4
    prompt: "Приведите аргументы за тезис: {motion}"
  - id: opponent
    agent: claude-3-opus
    prompt: "Приведите аргументы против тезиса: {motion}"
  - id: judge
    agent: gpt-4-turbo
    prompt: |
      Даны аргументы:
      За: {proponent.output}
      Против: {opponent.output}
      Определите победителя и объясните решение.
    deps: [proponent, opponent]
output: judge.output
```

Оркестратор компилирует это описание в граф, разрешает плейсхолдеры и выполняет сценарий. Рекурсивные конструкции (например, «дебаты до сходимости») выражаются через макросы циклов, которые генерируют ациклические подграфы на каждой итерации.

5 Эксперименты

5.1 Постановка

Мы оценили GRA на трёх эталонных тестах:

- **MHQA** (Multi-Hop Question Answering): 500 вопросов из HotpotQA, требующих информации из нескольких документов.
- **CodeGen-Swarm**: 200 задач программирования из HumanEval и MBPP, решаемых роём «кодер–ревьюер–тестирующий».
- **DebateScore**: 100 тем для дебатов, оценённых коллегией из трёх LLM-судей.

Базовые методы включали одиночную лучшую LLM (GPT-4), статический последовательный конвейер и групповой чат AutoGen. Все эксперименты использовали одни и те же API LLM с температурой 0.7.

5.2 Результаты

В таблице 1 приведены основные итоги. GRA превосходит базовые методы на всех задачах, особенно в многозвенном QA, где динамическое ветвление позволяет рою параллельно исследовать разные цепочки доказательств.

Таблица 1: Точность / pass@1 (%) на трёх бенчмарках.

Метод	MHQA	CodeGen	Дебаты (% побед)
Одиночная LLM (GPT-4)	72.1	78.4	55.0
Статический конвейер	76.8	82.1	58.3
Групповой чат AutoGen	79.4	84.5	61.7
GRA	85.3	88.9	68.2

Измерения задержки показывают, что параллельное выполнение GRA сокращает время «от стены до стены» на 18% по сравнению с AutoGen на сложных задачах.

5.3 Абляционный анализ

Отключение динамического планировщика (принудительный топологический порядок) снижает точность MHQA до 80.1%, что подтверждает важность адаптации во время выполнения. Отключение резервных узлов резко снижает устойчивость при высоком уровне отказов (10% сбоев узлов ведут к падению доли успешных выполнений на 22%).

6 Заключение

Мы представили GRA — графовую архитектуру рассуждений, позволяющую строить гибкие и устойчивые рои LLM. Представляя сотрудничество в виде направленного ациклического графа и предлагая высокоуровневый декларативный язык, GRA упрощает проектирование сложных мультиагентных сценариев и одновременно повышает производительность за счёт динамического планирования и отказоустойчивости. В будущем мы планируем исследовать самооптимизирующиеся графы, интеграцию с внешними базами знаний и формальную верификацию поведения роёв.

Благодарности

Мы благодарим сообщество открытого исходного кода за инструменты, сделавшие этот проект возможным.

Список литературы

- [1] Y. Li et al., “Collaborative LLM agents,” *NeurIPS Workshop*, 2023.
- [2] C. Wang et al., “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation,” *arXiv:2308.08155*, 2024.
- [3] LangChain, <https://www.langchain.com>, 2024.
- [4] Microsoft AutoGen, <https://microsoft.github.io/autogen/>, 2024.
- [5] M. Yasunaga et al., “QA-GNN: Reasoning with Language Models and Knowledge Graphs for Question Answering,” *NAACL*, 2021.